## (a) Authoring pipeline

`pipeline/run.py` · one command · a step is skipped when its output exists

**book.epub**
public domain or rights-cleared

**games/<g>/**
libre starting game: open code and open art

**refs/*.png | brief**
1..n reference images and / or a text brief

**selection**
chapters to build, e.g. `c01 c02`

*repository*

*selection*

**1 Ingest**
zip → XHTML spine → headings and paragraphs; stable ids `c01.p0042`; `sha256` of the input

standard library only

**book.json**
+ `book.txt`, one `[id]` marker per paragraph

**2 Story**
`gemini-4-argon`
`thinking high · schema`
whole book in one context → ground truth S: characters, chapters with beats, items, lines; every element cites its source paragraphs (d1)

validate — *fail ⇒ exit 1*

**story.json**
+ `story.log.json`: interaction id, usage, problems

**3 Look and assets**

*format fixed by the game code (3a), subject by the book (3c), appearance by the look (3b); 3d fills inside those constraints*

**3a Slots**
`gemini-4-argon`
`thinking high · schema`
whole repository in context → every slot the loader reads: role, kind, required, count, where {files, code}, format (frame grid, animations, facings, tile size, colour value)

cached beside the clone by repo hash

ground — *claims vs real files (d2)*

**slots.json**
grounded slots + `rejected[]`; two synthetic slots: dialogue portrait, chapter backdrop

**3b Bible**
`gemini-4-argon`
`thinking medium · schema`
references + brief + slot kinds + engine notes → palette, prompt prefix, avoid list, one drawing rule per kind, UI colours

keyed by `sha1(brief, refs, kinds)`

blocklist — *franchise or artist ⇒ refuse*

**style.json**
palette · prefix · rules per kind · `ui` colours

**3c Cast**
`gemini-4-argon`
`thinking medium · schema`
subjects of the selection × slots → one assignment per (slot, subject): protagonist → player, every character → portrait, every chapter → backdrop, items → pickups, colour slots → palette values

*unknown ids dropped*

**cast.json**
assignments ⟨slot, subject, value, why⟩; keyed by selection

**3d Fill**
`gemini-nano-banana-2.1`
`1K · jpeg → png`
per assignment: image fitted to w × h, white keyed to alpha; sheet = one pose per facing, composed static on the frame grid, plus scaled copies; tilesets kept upstream; 4 workers; cached by prompt

fit format — *required failure ⇒ flat palette shape, listed missing*

**manifest.json**
fills[slot][subject] · replaces[upstream] · parameters · kept_upstream · unfilled · missing

**4 Voice**
`gemini-3.8-flash-tts`
`speech_config · style`
one clip per utterance: summaries, goals, beats, item meanings (narrator); lines plain + original (one pool voice per character, delivery from its voice sentence)

cached by `sha1(voice|text)`

**audio_manifest.json**
`assets/audio/<hash>.wav` per utterance; failures recorded

**5 Adapt**
`antigravity-preview-09-2026`
`managed agent · model gemini-4-argon`

**remote sandbox /workspace**
in: repository text files (< 400 kB, vendor dirs skipped), `story`, `slots`, `style`, `selection`, both manifests, list of upstream binaries, `CONTRACT.md` (20 points). No asset bytes.
out: game rewired and rewritten in place, `CHECKS.md`; environment tarball downloaded

install: copy → `builds/`, overlay `assets/`, keep the previous build · fallback: one direct call, structured output `{files[], checks}`

**CHECKS.md**
one line per contract point, written by the agent; `adapt.log.json`: interaction and environment ids, status, seconds

npm build — *fails ⇒ reject*

**G builds/<book>/**
`game/` static files, exposes `window.playtest`, records every session
`assets/` + manifests
`story.json` `slots.json` `style.json` `cast.json` `CHECKS.md` `adapt.log.json`
`verification/round_n/` critique, patch, before / after, `report.json`

no model call at play time

*G served as static files*

*yes: copy the changed files into G; `report.json` records every round, score before / after, kept or reverted*

## (c) Repair loop — `verify.py: repair_loop`, shared by (6) and (7)

**E evidence directory**
`shots/NNN_label.png` `events.jsonl` `result.json` {score, errors, steps}
round 0: `verification/round_0/before/` (6) or `verification/session_<ts>/` (7); round n: `round_n/after/`

*evidence in*

*round n ≤ N*

**Critique**
`gemini-4-argon`
`thinking medium · schema`
in: E (≤ 24 screenshots, numbered event log, result), S, `slots`, `style`, manifest, `CONTRACT.md`, game source without protected files
out: ≤ 8 defects by severity, each with evidence and search / replace fixes (d4)

*first defect with evidence ∧ fixes*

**Patch in staging**
copy of G without `node_modules` and sessions; `apply_fixes`: each search unique, protected files refused; `node_modules` symlinked

builds? — *no: reject, stop*

**Replay**
same scripted playthrough on the staging copy → E′ with s(G′) and err(G′); saved under `round_n/after/`

s(G′) ≥ s(G) ∧ no new error — *no: discard staging, stop*

*yes: E ← E′, s(G) ← s(G′), next round*

**Legend**
- generative model call
- deterministic code
- artifact on disk
- deterministic gate
- managed-agent sandbox
- data flow
- next round of the loop

*Every model call is followed by a deterministic gate; every step is cached and resumable; no model runs while the player plays.*

## (b) Observation

**7 Human play**
the player opens `game/` from any static server; recorder on by default → `localStorage`: position every 1 s, every action, every line played, console errors, canvas shot every 5 s; "Download session" link

**session.json**
uploaded by the player, offline

**watch.py · unpack_session**
data-URL shots → `shots/`, events → `events.jsonl`, errors → `result.json`; review focus: where it broke, where the player got stuck (> 60 s), where the player was confused

**6 Scripted playthrough**
`playtest.py`: serve G over local HTTP; headless Chromium via Playwright; wait for `window.playtest`; for each character of the selection: `goto`, `talk`, require `state()` to change; for each item: `goto`, `collect`, require it in `state()`; screenshot after every step; console and page errors; 90 s budget
→ E with s(G)

## (d) Definitions

**d1 Ground truth S (`story.json`, stage 2)**

characters: id, name, look, voice, [first, last] chapter
chapters: summary, goal, beats {plain, original, source_ids}
items: chapter, meaning, source_ids
lines: character, chapter, plain, original, source_ids

Invariants checked by `validate`, not by the model:
∀ source_id: exists in the book ∧ lies in the element's chapter
∀ chapter: |beats| ≥ 3; ∀ line: chapter within its speaker's range
∀ plain: |words| ≤ 14; ids unique; chapter order = book order
The adapted game shows these texts and nothing else in a character's voice.

**d2 Slot model (`slots.json`, stage 3a)**

slot = ⟨id, role, kind, required, count, where {files, code}, format⟩
role ∈ {player, npc, enemy, pickup, terrain, decoration, backdrop, portrait, ui, sound, music, font, parameter}; kind ∈ {sheet, tileset, image, model, material, colour, audio, font, text}

grounded iff every named file exists ∧ claimed size ∈ measured sizes ∧ sheet divisible by its frame ∧ animation rows and frames fit ∧ metadata file and code refs exist ∧ a colour value is found in code; else rejected

A fill is ⟨slot, subject⟩ → path, size, prompt; `replaces[upstream] = path`. The adapter rewires slots to fills, never draws, resizes or recolours (contract §12).

**d3 Test interface (`window.playtest`, contract §17)**

`characters()` `items()` → ids and positions
`goto(id)` teleport next to a character or item
`talk()` `collect()` interact with what is in front
`state()` chapter, beats seen, items collected, lines heard
`screenshot()` canvas as data URL

Playthrough score s(G) = number of passed steps: API present · each character of the selection present · each item present · talk changes `state()` · collect records the item · no console error
Evidence E = ⟨`shots/*.png`, `events.jsonl`, `result.json`⟩; err(G) = console and page errors

**d4 Patch acceptance (`verify.py`)**

defect d = ⟨kind, severity, evidence, observed, expected, fixes⟩, kind ∈ {crash, unreachable, stuck, story, accessibility, visual, rights}
evidence = ∅ ⇒ d is discarded: no screenshot, log line or story id, no defect

patch p = [⟨path, search, replace⟩] is applicable iff every `search` occurs exactly once in `path` and `path` ∉ {story, selection, manifests, LICENSE, NOTICE}

Keep p iff G′ = G ⊕ p builds ∧ s(G′) ≥ s(G) ∧ err(G′) ⊆ err(G)
One defect per round; at most 3 rounds (6) or 2 rounds (7); every before / after pair is kept.